

Contents

1	GW2 Progression — 系统架构与设计说明	2
1.1	1. 系统架构总览	2
1.2	2. 核心数据流	3
1.2.1	2.1 账号分析流 (Primary Flow)	3
1.2.2	2.2 制作计算流 (Crafting Flow)	4
1.2.3	2.3 估值计算流 (Valuation Flow)	5
1.3	3. 数据模型 (Pydantic)	5
1.3.1	3.1 核心模型	5
1.3.2	3.2 制作模型	5
1.3.3	3.3 目标与 Build 模型	5
1.4	4. 数据库设计 (SQLite)	6
1.4.1	4.1 表结构	6
1.4.2	4.2 连接管理	6
1.4.3	4.3 数据归档	6
1.5	5. API 端点清单 (40+)	6
1.5.1	5.1 账号分析	6
1.5.2	5.2 价值评估	7
1.5.3	5.3 制作	7
1.5.4	5.4 目标追踪	7
1.5.5	5.5 成长规划	7
1.5.6	5.6 Build 推荐	8
1.5.7	5.7 TP 策略	8
1.5.8	5.8 Agent 与系统	8
1.6	6. GW2 API 集成	8
1.6.1	6.1 已接入 API (22+)	8
1.6.2	6.2 缓存策略	9
1.7	7. 前端架构 (SPA)	9
1.7.1	7.1 模块结构	9
1.7.2	7.2 11 个 Tab 面板	9
1.8	8. 服务层详解 (22 Services)	10
1.8.1	8.1 核心服务	10
1.8.2	8.2 估值服务	10
1.8.3	8.3 制作服务	10
1.8.4	8.4 搜索与市场	10
1.8.5	8.5 目标与 Build	10
1.9	9. 安全与运维	11
1.9.1	9.1 安全	11
1.9.2	9.2 可观测性	11
1.9.3	9.3 部署	11
1.10	10. 测试架构	11
1.11	11. 功能实现清单	12
1.11.1	11.1 账号数据	12
1.11.2	11.2 账号估值	12
1.11.3	11.3 数据可视化	12
1.11.4	11.4 物品搜索	12
1.11.5	11.5 制作计算	12
1.11.6	11.6 交易策略	13
1.11.7	11.7 目标追踪	13
1.11.8	11.8 目标规划 (Phase 3)	13
1.11.9	11.9 Build 推荐	13
1.11.10	11.10 成长 Agent	13
1.11.11	11.11 基础设施	13
1.12	12. GitNexus 代码图谱指标	14
1.13	附录: PlantUML 图示源文件索引	14

1 GW2 Progression — 系统架构与设计说明

Version: 0.3.0

成熟度等级: A-

测试: 148 tests, 20 test files

代码图谱: 2,705 nodes, 5,957 edges (GitNexus)

PDF 图示说明: 原文中的 ASCII 架构图、流程图和目录树已重绘为 PlantUML 图示并渲染嵌入。

1.1 1. 系统架构总览

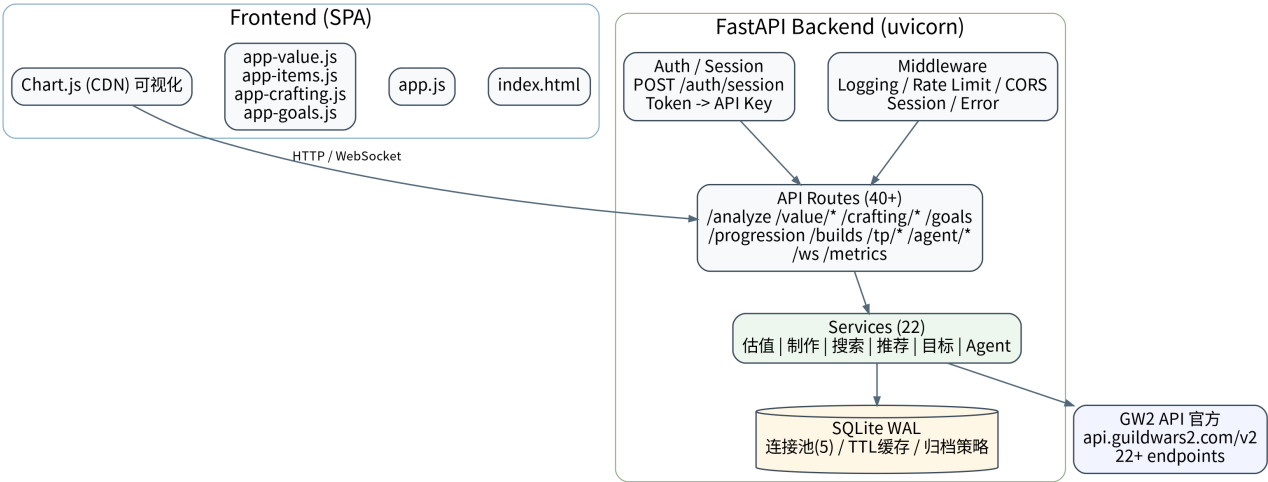


图 1. 系统架构总览 - PlantUML 重绘渲染图

1.2 2. 核心数据流

1.2.1 2.1 账号分析流 (Primary Flow)

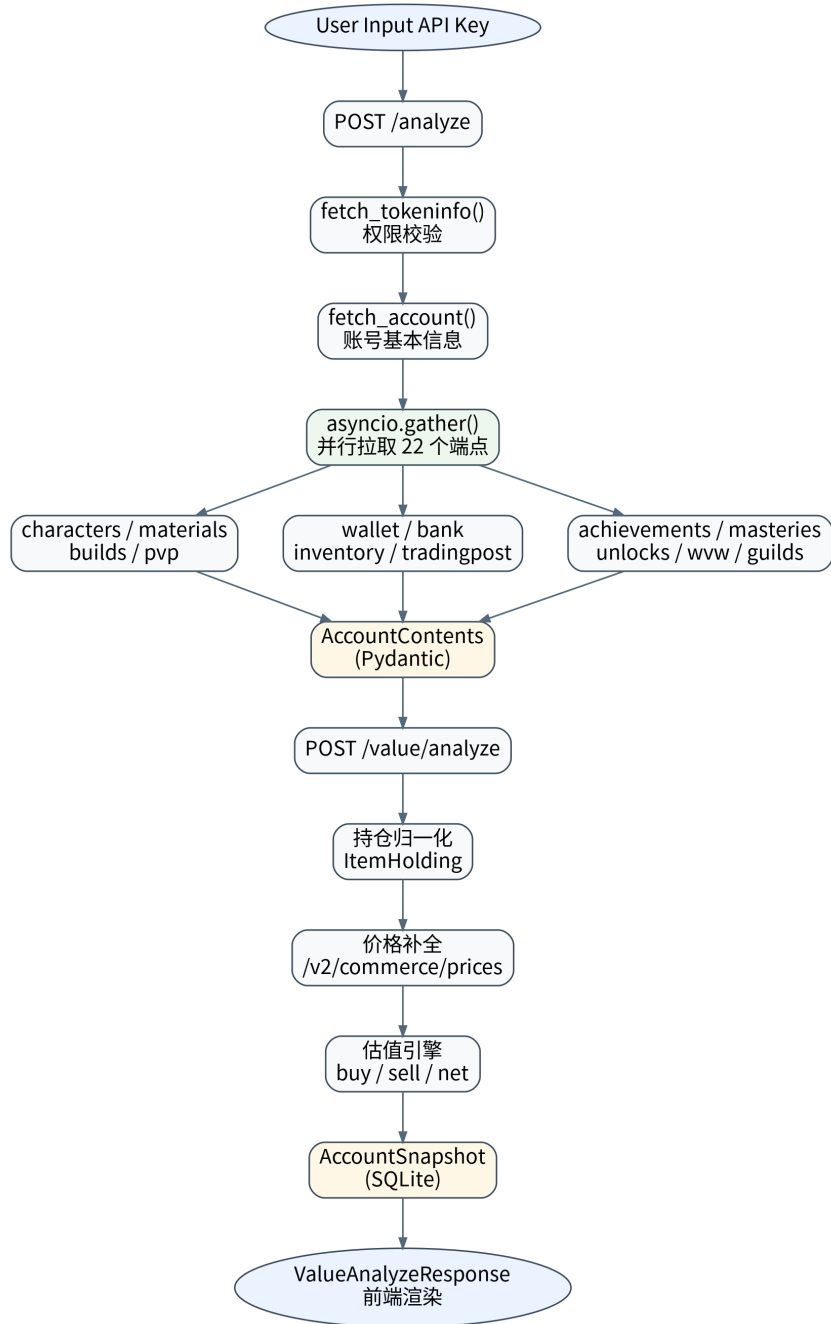


图 2. 账号分析流 Primary Flow - PlantUML 重绘渲染图

1.2.2 2.2 制作计算流 (Crafting Flow)

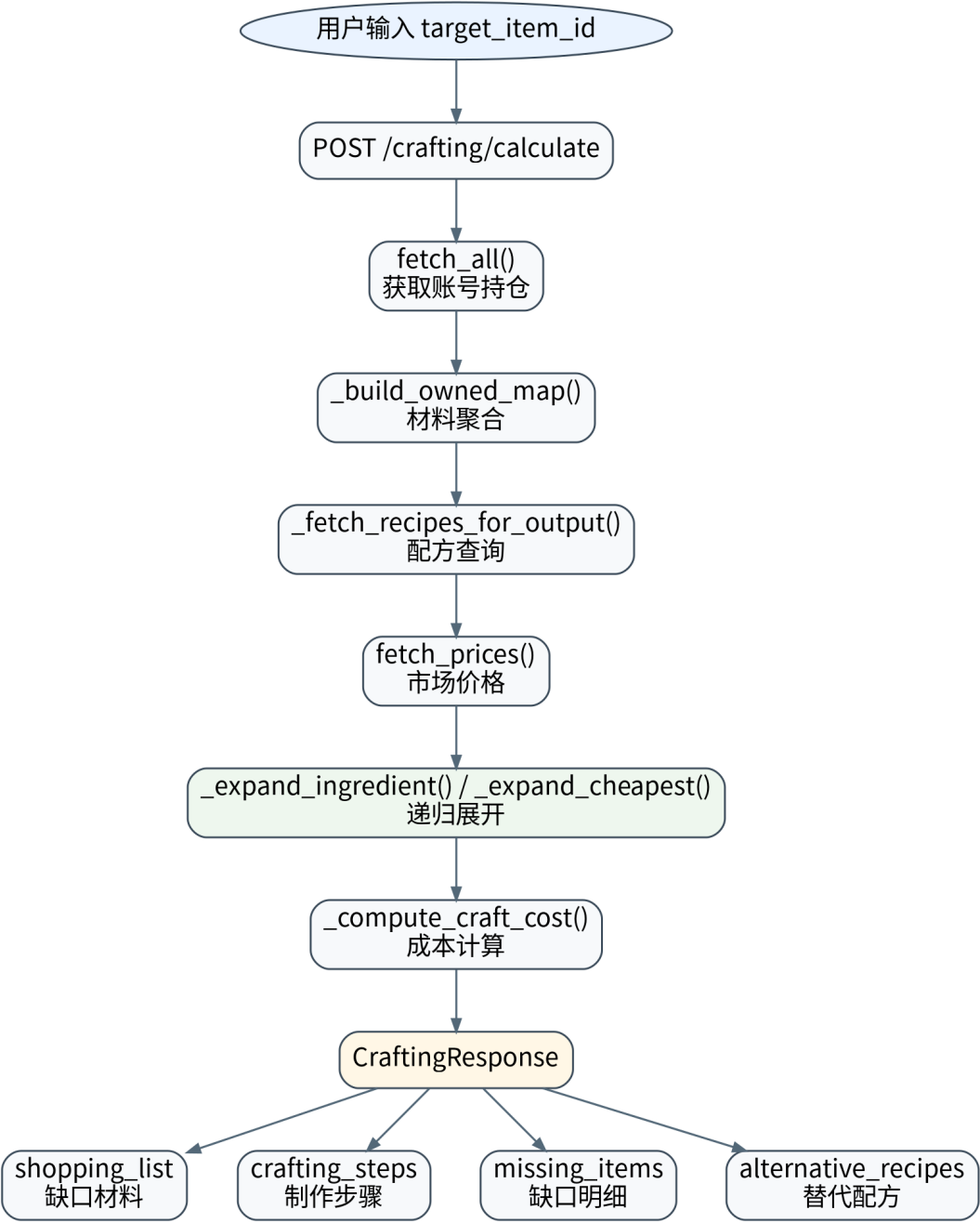


图 3. 制作计算流 Crafting Flow - PlantUML 重绘渲染图

1.2.3 2.3 估值计算流 (Valuation Flow)

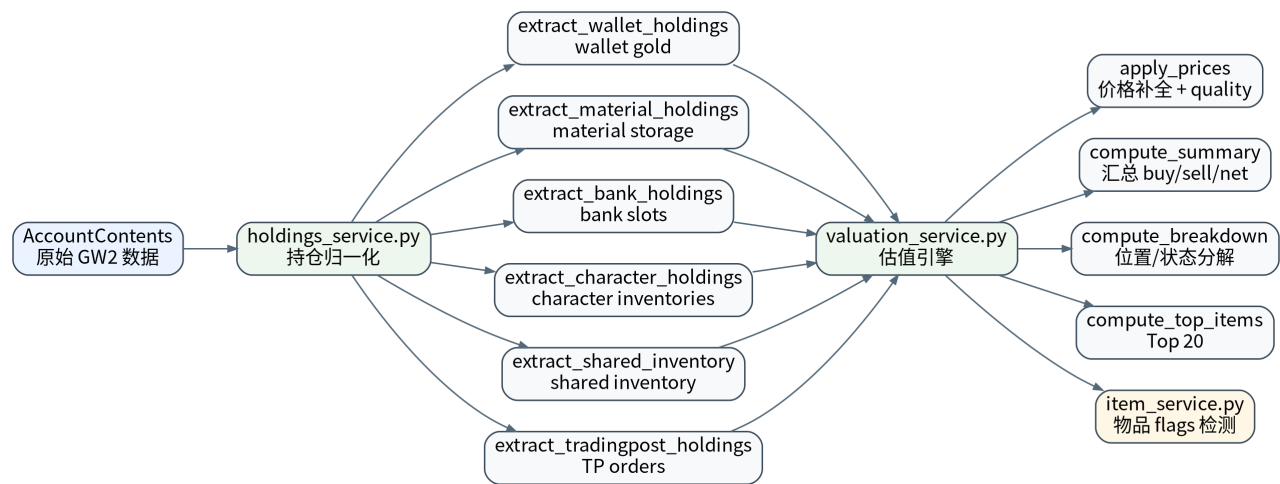


图 4. 估值计算流 Valuation Flow - PlantUML 重绘渲染图

1.3 3. 数据模型 (Pydantic)

1.3.1 3.1 核心模型

模型	用途	关键字段
AccountContents	原始 GW2 数据容器	32 字段, errors dict
ItemHolding	归一化持仓	item_id, count, location, price_buy/sell, quality_status
PriceData	价格快照	buy/sell unit_price + quantity
ValueSummary	价值汇总	total_value_buy/sell, wallet/materials/bank, reliable/risky
ValueBreakdown	价值分解	by_location[], by_status[]
TopItem	Top 物品	item_id, count, value_buy/sell
ItemValueDelta	物品价值变化	old/new count+price, primary_cause
AccountValueDelta	账号价值变化	total_delta, price_effect, top_gainers/decliners

1.3.2 3.2 制作模型

模型	用途
CraftIngredient	制作材料 (含 owned/missing)
CraftStep	制作步骤 (含 disciplines/rating)
CraftingResponse	计算结果 (shopping_list, steps)
CraftingPlanLine	计划行 (required_count, owned, missing)
CraftingPlanResult	制作计划 (craft_vs_buy_delta)
RecipeDecision	优化决策 (buy/craft/use_owned)
RecipeOptimizationResult	优化结果 (多策略)

1.3.3 3.3 目标与 Build 模型

模型	用途
TrackedGoal	用户追踪目标
ProgressionGoalTemplate	传奇/升华模板
GoalRequirement	模板需求项
GoalRequirementStatus	需求完成状态
GoalPlan	目标计划 (completion%)
BuildTemplate	Build 模板 (20 curated)
AccountBuildReadiness	Build 可达性评分
TradingPostSignal	TP 交易信号
ProtectedAsset	受保护资产
ProgressionAdvice	成长建议

1.4 4. 数据库设计 (SQLite)

1.4.1 4.1 表结构

表名	用途	关键索引
account_snapshots	账号快照	(account_name)
item_holdings	持仓明细	(snapshot_id)
price_snapshots	价格快照	(item_id)
account_value_history	价值历史	(account_name)
valuation_warnings	估值警告	(snapshot_id)
static_items	静态物品数据	(id)
static_recipes	静态配方	(output_item_id)
recipe_ingredients	配方材料	(recipe_id, item_id)
tracked_goals	用户目标	(account_name)
progression_goal_templates	目标模板	(template_id)
goal_requirements	模板需求	(template_id)
protected_assets	受保护资产	(account_name, item_id)

1.4.2 4.2 连接管理

```
# 连接池: asyncio.Queue + 5 连接复用
async with using_db() as db:
    cursor = await db.execute("SELECT ...")
    rows = await cursor.fetchall()
    # 自动 commit
# 自动 release 回池
```

1.4.3 4.3 数据归档

```
SNAPSHOT_RETENTION = 20 # 保留最近 20 快照
PRICE_RETENTION = 7 # 价格保留 7 天
HISTORY_RETENTION = 90 # 历史保留 90 天
```

1.5 5. API 端点清单 (40+)

1.5.1 5.1 账号分析

方法	路径	用途
POST	/analyze	拉取账号全量数据
POST	/resolve	代理 GW2 静态数据

1.5.2 5.2 价值评估

方法	路径	用途
POST	/value/analyze	全量价值分析
GET	/value/items/search?q=	物品搜索 (名称/ID)
GET	/value/items/{id}/detail	物品详情
GET	/value/items/locations	物品位置
GET	/value/items/high-value	高价值物品
GET	/value/items/unpriced	未定价物品
GET	/value/items/account-bound	账号绑定物品
GET	/value/delta	快照差分
GET	/value/top-gainers	增值 Top
GET	/value/top-decliners	减值 Top
GET	/value/listings/{id}	TP 订单簿
POST	/value/listings/batch	批量订单簿
POST	/value/cleanup	数据归档
GET	/value/{id}/detail	价值详情

1.5.3 5.3 制作

方法	路径	用途
POST	/crafting/calculate	制作计算
POST	/crafting/calculate/cheapest	最优配方计算
POST	/crafting/plan	制作计划 (craft_vs_buy)
POST	/crafting/optimize	多策略优化
POST	/crafting/refresh/items	刷新物品数据
POST	/crafting/refresh/recipes	刷新配方数据
GET	/crafting/recipes/by-output/{id}	配方查询
GET	/crafting/refresh/progress/{task_id}	刷新进度

1.5.4 5.4 目标追踪

方法	路径	用途
POST	/goals	创建目标
GET	/goals	目标列表
GET	/goals/{id}	目标详情
POST	/goals/{id}/refresh	刷新进度
DELETE	/goals/{id}	删除目标

1.5.5 5.5 成长规划

方法	路径	用途
GET	/progression/templates	目标模板列表
GET	/progression/templates/{id}	模板详情
POST	/progression/plans	生成规划

1.5.6 5.6 Build 推荐

方法	路径	用途
GET	/builds/templates	Build 模板列表
GET	/builds/templates/{id}	Build 详情
POST	/builds/recommendations	Build 推荐
POST	/builds/readiness/{id}	Build 可达性

1.5.7 5.7 TP 策略

方法	路径	用途
GET	/tp/signals	交易信号
GET	/tp/sell-candidates	可卖资产
GET	/tp/buy-candidates	建议购买
GET	/tp/protected-assets	受保护资产
POST	/tp/protected-assets	保护资产
DELETE	/tp/protected-assets/{id}	解除保护

1.5.8 5.8 Agent 与系统

方法	路径	用途
POST	/agent/progression/advice	成长建议
POST	/agent/progression/weekly-plan	周计划
GET	/health	健康检查
GET	/metrics	监控指标
GET	/ws	WebSocket
POST	/auth/session	Session 创建

1.6 6. GW2 API 集成

1.6.1 6.1 已接入 API (22+)

端点	权限	用途
/v2/tokeninfo	无	API key 验证
/v2/account	account	账号基本信息
/v2/characters	characters	角色数据
/v2/account/wallet	wallet	钱包货币
/v2/account/bank	inventories	银行
/v2/account/materials	inventories	材料库
/v2/account/inventory	inventories	共享背包
/v2/account/achievements	progression	成就
/v2/account/masteries	progression	专精
/v2/account/mastery/points	progression	专精点数
/v2/account/buildstorage	builds	Build 存储
/v2/pvp/stats	pvp	PvP 统计
/v2/pvp/games	pvp	PvP 对局
/v2/pvp/standings	pvp	PvP 天梯
/v2/commerce/transactions/current/buys	tradingpost	当前买单
/v2/commerce/transactions/current/sells	tradingpost	当前卖单

端点	权限	用途
/v2/commerce/prices	无	市场价格
/v2/commerce/listings	无	订单簿
/v2/account/skins	unlocks	已解锁皮肤
/v2/account/dyes	unlocks	已解锁染料
/v2/account/minis	unlocks	已解锁迷你
/v2/account/finishers	unlocks	已解锁终结技
/v2/items	无	物品信息
/v2/recipes	无	配方信息
/v2/recipes/search	无	配方搜索
/v2/search	无	物品/名称搜索

1.6.2 6.2 缓存策略

缓存	位置	TTL	大小
市场价格	内存	15 min	2,000 items
订单簿	内存	5 min	1,000 items
物品 flags	TTL cache	24 h	4,096 items
物品名称	TTL cache	24 h	4,096 items
配方	TTL cache	24 h	2,048 items
静态数据	SQLite	手动刷新	50k+ items

1.7 7. 前端架构 (SPA)

1.7.1 7.1 模块结构

static/	
—— index.html	# 入口 (11 Tabs)
—— style.css	# 全局样式 + 响应式
—— app.js	# 核心: 缓存, 解析, tab切换, analyze流
—— app-value.js	# Value Dashboard: 图表, 持仓, 材料, delta
—— app-items.js	# 物品搜索: 名称搜索, 位置钻取, TP深度
—— app-crafting.js	# 制作计算: 配方搜索, 结果展示, 替代配方
—— app-goals.js	# 目标追踪: 创建, 进度, 刷新, 删除

1.7.2 7.2 11 个 Tab 面板

Tab	ID	功能
Overview	tab-overview	账号卡片, 权限网格, 错误显示
Value	tab-value	总价值, 图表, Top物品, 持仓, 变化
Characters	tab-characters	角色选择, 纸娃娃, 装备列表
Wardrobe	tab-wardrobe	皮肤搜索/过滤/分页
Wallet	tab-wallet	货币列表 (金币格式化)
Inventory	tab-inventory	材料, 银行, 共享背包
Progression	tab-progression	专精, 专精点, 成就
PvP	tab-pvp	统计, 对局, 天梯
Unlocks	tab-unlocks	皮肤/染料/迷你/终结技
WvW	tab-www	WvW 等级, 队伍
Builds	tab-builds	装备模板, Build 模板
Items	tab-items	物品搜索, 详情, 市场深度
Crafting	tab-crafting	制作计算, 替代配方

Tab	ID	功能
Goals	tab-goals	目标创建, 进度追踪

1.8 8. 服务层详解 (22 Services)

1.8.1 8.1 核心服务

Service	职责	关键函数
gw2_client.py analyzer.py cache.py auth_service.py price_service.py	GW2 HTTP 客户端 分析编排 TTL 缓存 Session 管理 价格服务	_get(), 22 fetch_* 函数 fetch_all(), AccountContents TTLCache, @cached create_session(), get_session() fetch_prices(), compute_price_quality(), warmup_price_cache()

1.8.2 8.2 估值服务

Service	职责	关键函数
holdings_service.py valuation_service.py	持仓归一化 估值引擎	6 extract_*_holdings() apply_prices(), compute_summary(), compute_breakdown()
item_service.py snapshot_service.py delta_service.py	物品 flags 快照编排 变化分析	is_account_bound(), get_item_flags() run_full_analysis() compare_snapshots()

1.8.3 8.3 制作服务

Service	职责	关键函数
recipe_service.py	配方引擎	calculate(), calculate_cheapest(), _expand_ingredient(), _expand_cheapest()
crafting_plan_service.py recipe_optimizer.py static_data_service.py	制作计划 配方优化 静态数据	create_plan() optimize_item(), optimize() refresh_items(), refresh_recipes(), find_recipes_by_output()

1.8.4 8.4 搜索与市场

Service	职责	关键函数
item_search_service.py	物品搜索	search_items_by_name(), get_item_detail(), get_filtered_items()
listing_service.py tp_strategy_service.py	订单簿 TP 策略	fetch_listings(), analyze_depth() generate_signals(), protect_asset()

1.8.5 8.5 目标与 Build

Service	职责	关键函数
goal_service.py	目标追踪	create_goal(), refresh_goal()
progression_service.py	成长规划	generate_goal_plan(), seed_templates()
build_service.py	Build 系统	calculate_readiness(), get_recommendations()
agent_service.py	Agent	generate_advice(), generate_weekly_plan()

1.9 9. 安全与运维

1.9.1 9.1 安全

- API 速率限制: 30 req/min per IP (429 响应)
- CORS: 全开 (开发阶段)
- Session Token: secrets.token_hex(24), 1h TTL
- API Key: 仅内存传输, 不记录日志
- 请求 ID 追踪: X-Request-ID header

1.9.2 9.2 可观测性

- 结构化日志: logging + request ID
- /metrics 端点: uptime, 请求/分析/错误计数
- WebSocket /ws: 实时通知推送
- 错误隔离: 每个 GW2 endpoint 独立 try/catch

1.9.3 9.3 部署

- Docker: python:3.12-slim
- Docker Compose: 端口映射 + 数据卷
- CI: GitHub Actions (ruff + pytest)
- 数据库: SQLite WAL, 自动建表迁移

1.10 10. 测试架构

```
tests/
|—— test_analyzer.py      # 7 tests — 分析器编排
|—— test_cache.py        # 6 tests — TTL 缓存
|—— test_gw2_client.py   # 15 tests — HTTP 客户端
|—— test_resolve.py      # 5 tests — 解析代理
|—— test_routes.py       # 8 tests — 路由层
|—— test_valuation.py    # 21 tests — 估值引擎
|—— test_crafting.py     # 15 tests — 配方引擎
|—— test_crafting_plan.py # 5 tests — 制作计划
|—— test_delta.py        # 8 tests — 快照差分
|—— test_item_service.py # 4 tests — 物品 flags
|—— test_item_search.py  # 5 tests — 物品搜索
|—— test_price_quality.py # 8 tests — 价格质量
|—— test_static_data.py  # 3 tests — 静态数据
|—— test_listings.py     # 4 tests — TP 订单簿
|—— test_goals.py        # 4 tests — 目标追踪
|—— test_progression.py  # 7 tests — 成长规划
|—— test_phase3.py       # 6 tests — Phase 3 模块
```

1.11 11. 功能实现清单

1.11.1 11.1 账号数据

- ☒ API Key 输入与验证
- ☒ Tokeninfo 权限检测
- ☒ 22 个 GW2 API 端点集成
- ☒ 权限门控 (每个 endpoint 按权限开关)
- ☒ 错误隔离 (每个 endpoint 独立 try/catch)
- ☒ 数据懒加载 (wardrobe 分页 200)

1.11.2 11.2 账号估值

- ☒ 6 种持仓归一化 (wallet/materials/bank/character/shared/TP)
- ☒ 市场价格补全 (/v2/commerce/prices)
- ☒ 三口径估值 (Instant Sell / Listing / Net Sell)
- ☒ 15% TP 手续费计算
- ☒ 不可交易物品标记 (account-bound/unpriced)
- ☒ 物品 flags 检测 (AccountBound/SoulbindOnAcquire)
- ☒ 价格质量评分 (reliable/illiquid/wide_spread)
- ☒ 流动性评分 (high/medium/low/illiquid)
- ☒ 套利检测 (buy/sell spread + fees)
- ☒ 市场深度 (top 5 buy/sell volume)

1.11.3 11.3 数据可视化

- ☒ 总览摘要卡片 (10 项)
- ☒ 价值摘要卡片 (8 项)
- ☒ 资产构成饼图 (Chart.js)
- ☒ 位置价值柱图 (Chart.js)
- ☒ 价值趋势折线图 (Chart.js)
- ☒ Top 20 Valuable Items 表格
- ☒ 金币格式化 Xg Ys Zc
- ☒ vs 前次快照对比 (▲/▼ delta)
- ☒ 多口径估值展示

1.11.4 11.4 物品搜索

- ☒ 按物品 ID 搜索
- ☒ 按物品名称搜索 (/v2/search)
- ☒ 按位置过滤 (6 种)
- ☒ 按状态过滤 (3 种)
- ☒ 快速筛选 (High Value/Unpriced/Bound)
- ☒ 物品详情面板
- ☒ 物品位置钻取
- ☒ 市场深度面板 (订单簿)

1.11.5 11.5 制作计算

- ☒ 目标物品输入 (名称搜索 + ID)
- ☒ Recipe tree 展开 (递归 3 层)
- ☒ 已有材料自动抵扣
- ☒ 缺口材料输出

- ☒ Shopping list
- ☒ Crafting steps
- ☒ 替代配方展示
- ☒ 最优配方路径 (多配方成本对比)
- ☒ 5 种优化策略 (cheapest/fastest/use_owned/preserve/minimize_gold)
- ☒ 循环检测 (visited set)
- ☒ 职业/等级过滤 (crafting discipline + rating)

1.11.6 11.6 交易策略

- ☒ Sell candidate 检测
- ☒ Buy candidate 检测
- ☒ Goal-protected asset
- ☒ 高 spread 警告
- ☒ 低流动性警告
- ☒ 手动资产保护

1.11.7 11.7 目标追踪

- ☒ 创建目标
- ☒ 目标列表
- ☒ 目标刷新 (重新计算进度)
- ☒ 删除目标
- ☒ 完成百分比
- ☒ 剩余成本估计

1.11.8 11.8 目标规划 (Phase 3)

- ☒ 8 个手工目标模板 (5 传奇 + 2 升华 + 1 背饰)
- ☒ Requirement 系统 (item/currency/achievement)
- ☒ 目标计划生成
- ☒ 持仓/钱包/成就映射
- ☒ 阻塞项检测

1.11.9 11.9 Build 推荐

- ☒ 20 个手工 curated Build (SnowCrows + MetaBattle)
- ☒ 9 职业覆盖
- ☒ Readiness Score (装备50%+职业30%+匹配20%)
- ☒ 缺口装备检测
- ☒ 缺口成本估计

1.11.10 11.10 成长 Agent

- ☒ 聚合分析 (目标 + Build + TP 信号)
- ☒ 行动建议生成
- ☒ 7 天周计划
- ☒ Goal-protected 资产保护

1.11.11 11.11 基础设施

- ☒ FastAPI 异步框架
- ☒ SQLite + WAL 模式
- ☒ DB 连接池 (5 连接)
- ☒ Session 管理
- ☒ API 速率限制
- ☒ Prometheus /metrics

- ☒ WebSocket 通知
- ☒ Docker 部署
- ☒ GitHub Actions CI
- ☒ 前端模块化 (5 JS 文件)
- ☒ 响应式设计 (768px/480px)

1.12 12. GitNexus 代码图谱指标

指标	数值
代码节点	2,705
关系边	5,957
功能社区 (Clusters)	84
执行流程 (Flows)	240
平均社区内聚度	0.75
最高内聚社区	Tests (1.0), Gw2_progression (0.92)
核心调用节点	fetch_all (22), run_full_analysis (17), resolve (12)
源文件	47 Python + 5 JS + HTML/CSS
测试文件	19 Python

1.13 附录：PlantUML 图示源文件索引

本 PDF 中渲染的图示对应以下 PlantUML 源文件：

图示	PlantUML 源文件
系统架构总览	puml/system_architecture.puml
账号分析流	puml/primary_flow.puml
制作计算流	puml/crafting_flow.puml
估值计算流	puml/valuation_flow.puml
前端模块结构	puml/frontend_modules.puml
测试架构	puml/test_architecture.puml